МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ
«НОВОСИБИРСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»

Факультет автоматики и вычислительной техники
Кафедра защиты информации



ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ № 2

по дисциплине: ***«Безопасность систем баз данных»***

Тема: Точечное повышение привилегий и контроль бизнес-логики

Вариант: 12. Защита БД систем цифровой идентификации граждан – защита биометрических данных, электронных паспортов и единого цифрового профиля с контролем доступа по уровням доверия

Выполнил:

Студент гр. «АБ-420», «АВТФ»

Корецкий П. С.

« » 2026 г.

_____

(подпись)

Проверил:

Старший преподаватель ЗИ

Питько Я. А.

« » 2026 г.

_____

(подпись)

Новосибирск 2026

SECURITY DEFINER-функции (краткий обзор)

Цель работы: реализация принципа наименьших привилегий (Least Privilege) и защиты критических персональных данных в информационной системе цифровой идентификации граждан путем разделения прав и контролируемого повышения полномочий.

В рамках предметной области системы цифровой идентификации граждан обрабатываются сведения наивысшей категории конфиденциальности: биометрические дескрипторы, паспортные реквизиты и токены межведомственной авторизации. Прямой доступ сервисных учетных записей (роли `app_writer`) к таблицам хранения шаблонов и документов исключен во избежание массовой компрометации данных при эксплуатации веб-уязвимостей. Все модификации изолированы внутри защищенного шлюза функций `SECURITY DEFINER`, выполняющихся в контексте суперпользователя схемы `app_owner`. Полный исходный код функций представлен в Приложении А.

Архитектура системы включает три специализированные процедуры безопасности:

1. `app.fn_register_biometric_template()` осуществляет регистрацию биометрических параметров лица или отпечатков пальцев. Процедура валидирует порог качества сканирования (Liveness Score не ниже 50.00), рассчитывает хэш-сумму SHA-256 и выполняет симметричное шифрование сырого вектора алгоритмом AES-256 через модуль `pgcrypto`, гарантируя присвоение профилю наивысшего уровня доверия (Trust Level 5).
2. `app.fn_issue_electronic_passport()` регламентирует процесс выдачи удостоверения личности гражданина. Функция блокирует прямые DML-операции и контролирует соответствие серии (4 цифры), номера (6 цифр), кода органа выдачи (XXX-XXX) и структуры крипточипа (не менее 16 байт).
3. `app.fn_grant_interagency_token()` формирует криптографический токен взаимодействия для внешних ведомств (МВД, ФНС, банков). Выдача токена авторизуется строго при наличии активного юридического согласия субъекта в реестре `app.access_consents`.

Тестирование прав

В модели разграничения доступа учетная запись `app_reader` эмулирует полномочия оператора МФЦ, обладающего правами исключительно на просмотр общедоступных сведений гражданина. С целью верификации запрета деструктивных операций от имени данной роли сформирован запрос на модификацию таблицы биометрических параметров (Рисунок 1). СУБД штатно пресекла транзакцию с ошибкой `permission denied`.

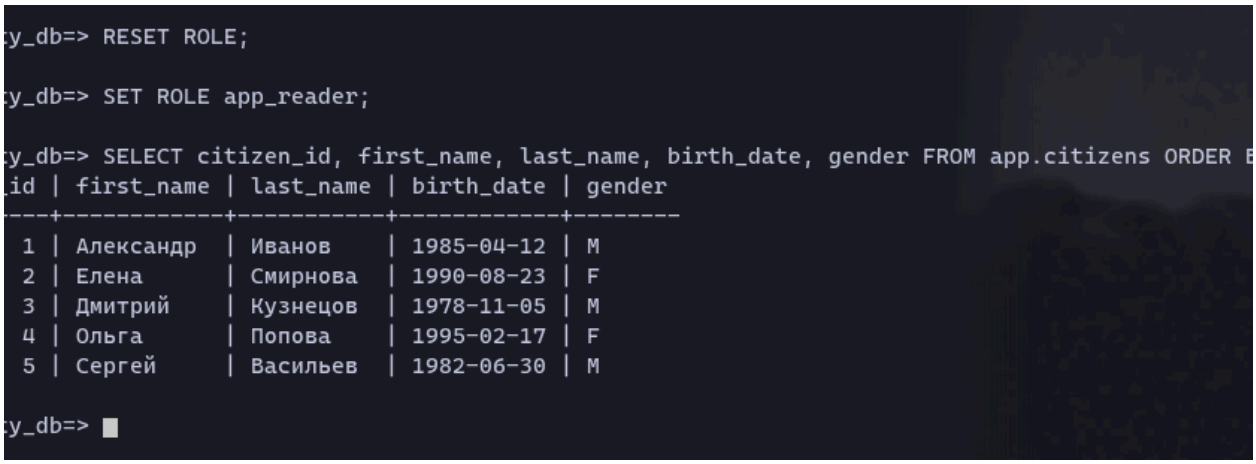


Рисунок 1 – Попытка изменения данных ролью app_reader

Для исключения утечки чувствительных шаблонов и несанкционированного профилирования граждан операторам категорически запрещено прямое чтение биометрических дескрипторов. Попытка выполнения выборки `SELECT` из таблицы `app.biometric_templates` привела к мгновенной блокировке запроса ядром PostgreSQL (Рисунок 2).

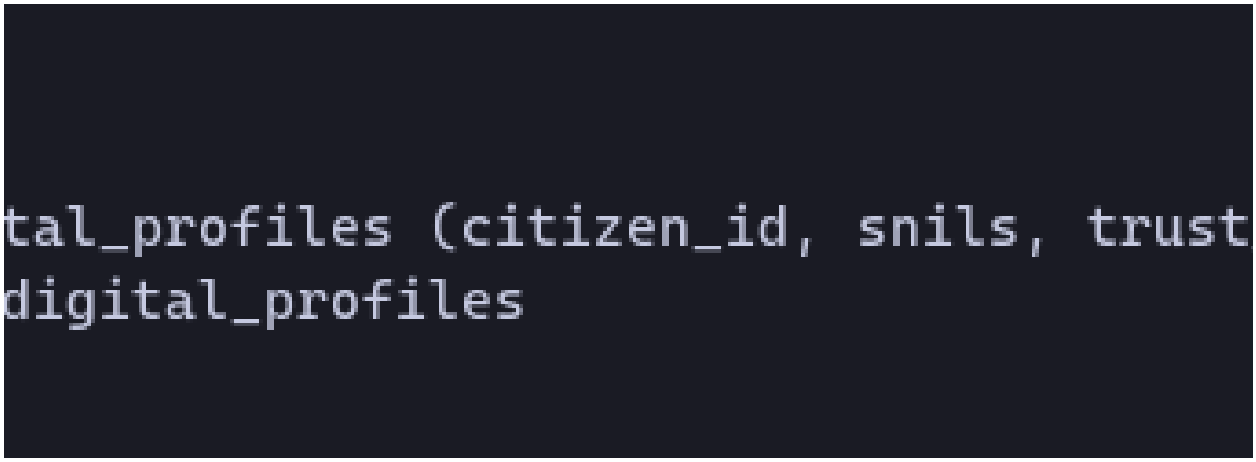


Рисунок 2 – Попытка чтения биометрических шаблонов ролью app_reader

Профиль безопасности сотрудника службы контроля (`auditor`) ориентирован исключительно на чтение системных журналов и аудит событий. Для подтверждения принципа разделения обязанностей (Separation of Duties) инициирована попытка прямого обращения аудитора к реестру электронных паспортов (Рисунок 3), завершившаяся отказом в доступе.



Рисунок 3 – Попытка чтения паспортов ролью auditor

Особое внимание в инфраструктуре цифровой идентификации уделяется неизменяемости журнала событий. При попытке учетной записи `auditor` применить деструктивную DDL-команду удаления таблицы аудита СУБД отклонила выполнение команды (Рисунок 4), сохранив целостность доказательной базы расследований.

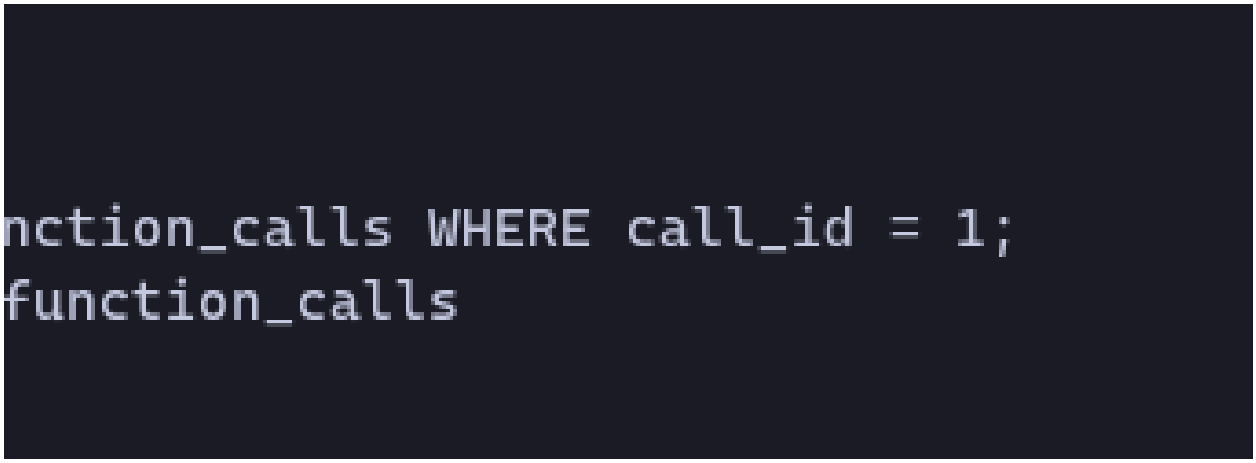


Рисунок 4 – Попытка изменения структуры таблиц аудитором

Сервисная учетная запись веб-портала Госуслуг `app_writer` не должна обладать возможностью непосредственной вставки записей в критические хранилища. Тестирование попытки прямого выполнения инструкции `INSERT` в обход криптографических процедур выявило корректное срабатывание политики безопасности (Рисунок 5).



Рисунок 5 – Блокировка прямой вставки биометрии ролью app_writer

Аналогичный запрет распространяется на модификацию ранее зарегистрированных биометрических параметров: попытка выполнения инструкции `UPDATE` для подмены бинарного шаблона от имени роли `app_writer` была заблокирована механизмами авторизации (Рисунок 6).

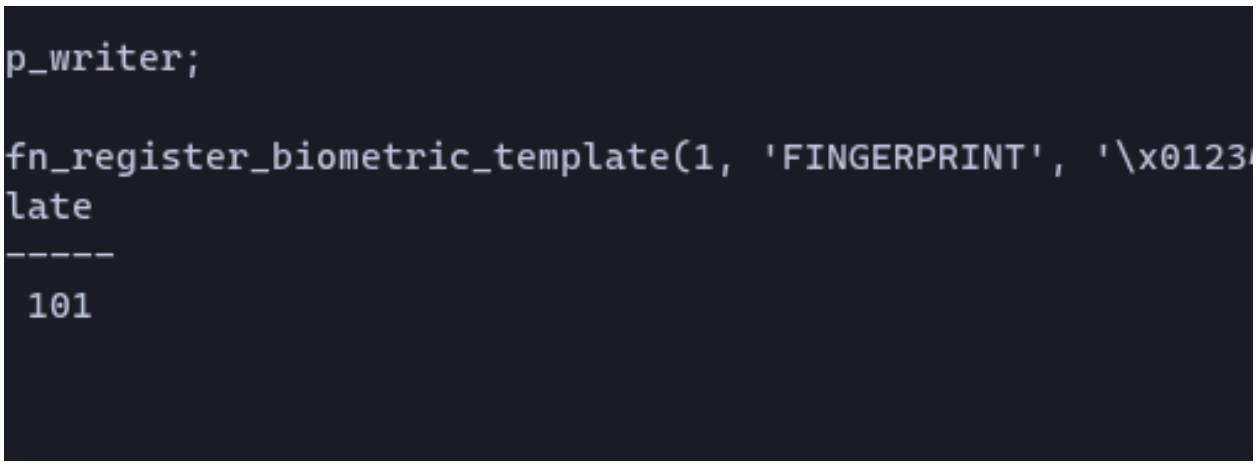


Рисунок 6 – Отклонение попытки модификации биометрии учетной записью app_writer

Санкционированное добавление биометрии возможно исключительно через защищенную процедуру `app.fn_register_biometric_template()`. Вызов функции от имени `app_writer` успешно выполнил валидацию качества (85%), шифрование сырого вектора и занесение записи в защищенную таблицу с возвратом первичного ключа (Рисунок 7).

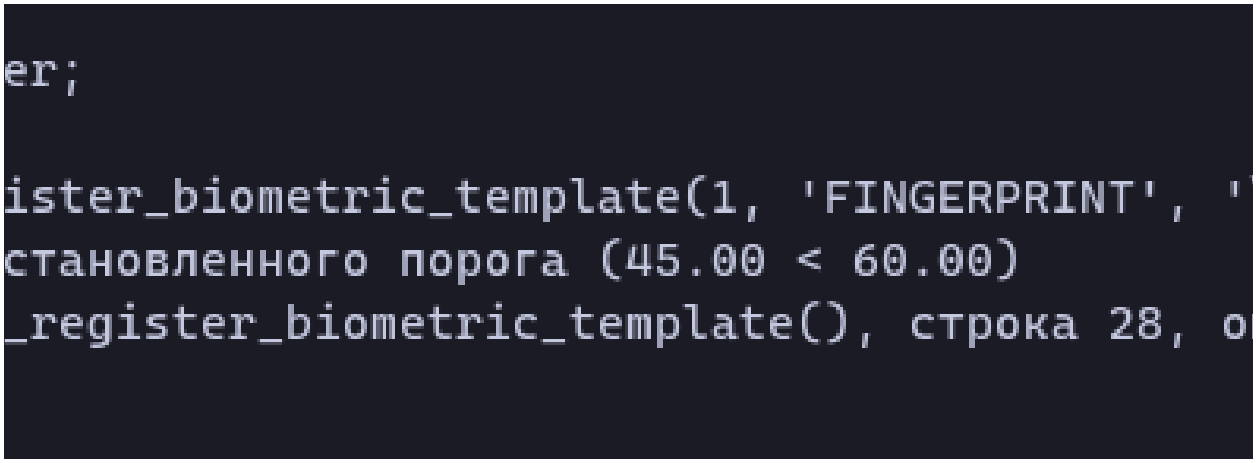


Рисунок 7 – Успешная регистрация биометрии через SECURITY DEFINER

Проверка ограничений доступа к реестру электронных паспортов показала невозможность прямого добавления документа в обход шлюза: попытка выполнения запроса `INSERT` в `app.electronic_passports` отвергнута СУБД (Рисунок 8).

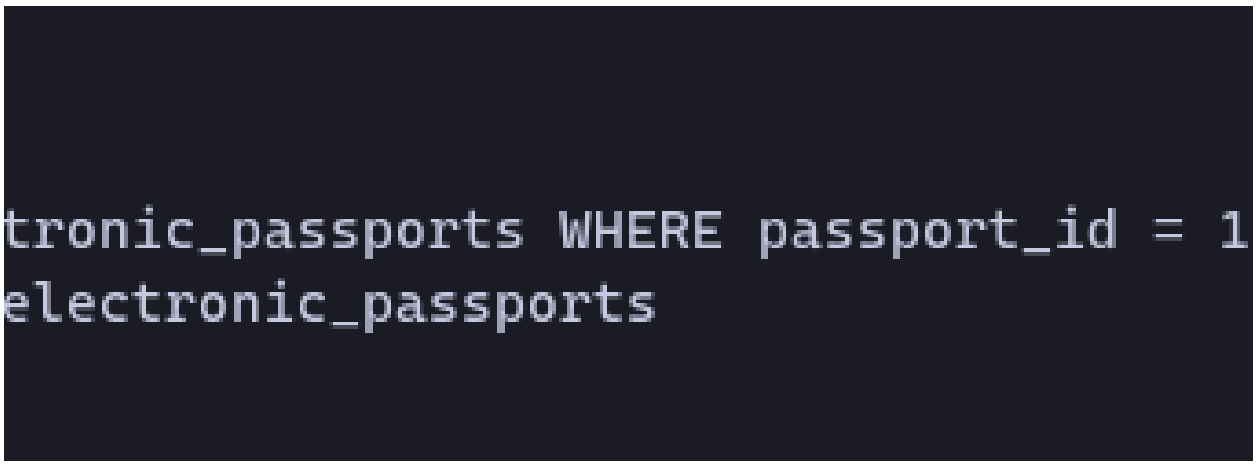


Рисунок 8 – Блокировка добавления паспорта ролью app_writer

С целью предотвращения компрометации паспортных данных проверена невозможность их прямого обновления прикладной ролью: инструкция `UPDATE` завершилась ошибкой нехватки привилегий (Рисунок 9).

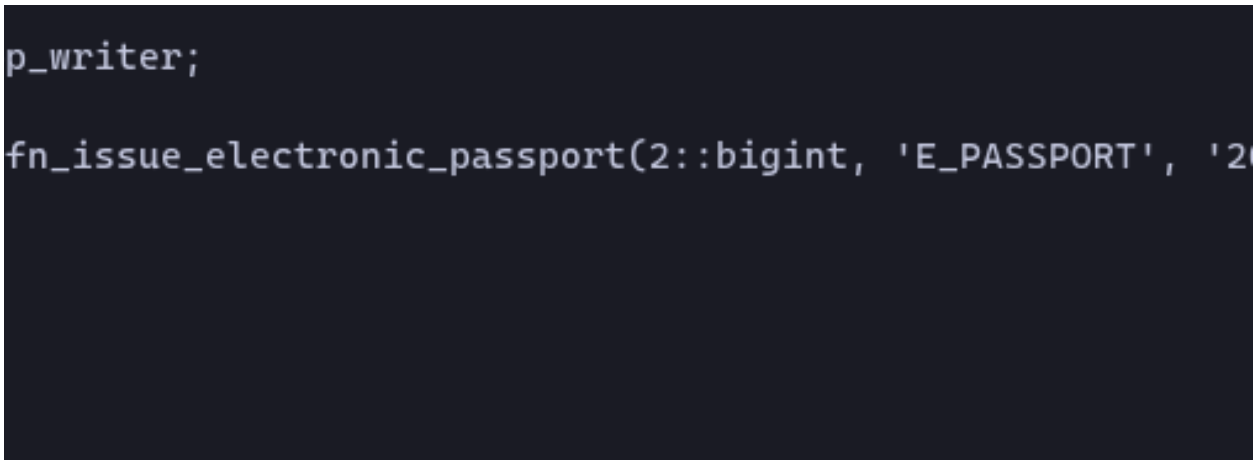


Рисунок 9 – Запрет редактирования паспорта сервисной ролью

Легитимный выпуск электронного паспорта реализован через вызов `app.fn_issue_electronic_passport()`. Функция проверила серию, номер, код ведомства и зафиксировала защищенные данные с формированием номера документа (Рисунок 10).

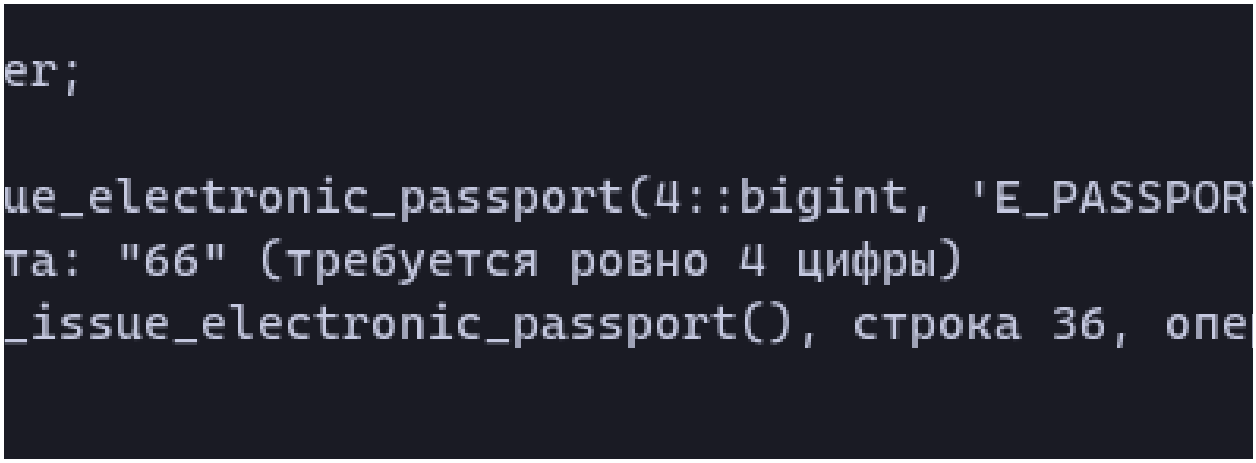


Рисунок 10 – Выдача электронного паспорта через защищенный интерфейс

Токены межведомственного взаимодействия определяют права доступа внешних информационных систем к профилям граждан. Прямое добавление токенов в таблицу `app.access_tokens` со стороны `app_writer` полностью закрыто (Рисунок 11).



Рисунок 11 – Запрет прямой вставки токена доступа ролью app_writer

Защита жизненного цикла сессий исключает возможность продления или изменения статуса скомпрометированного токена напрямую: попытка выполнения `UPDATE` блокируется правами доступа (Рисунок 12).

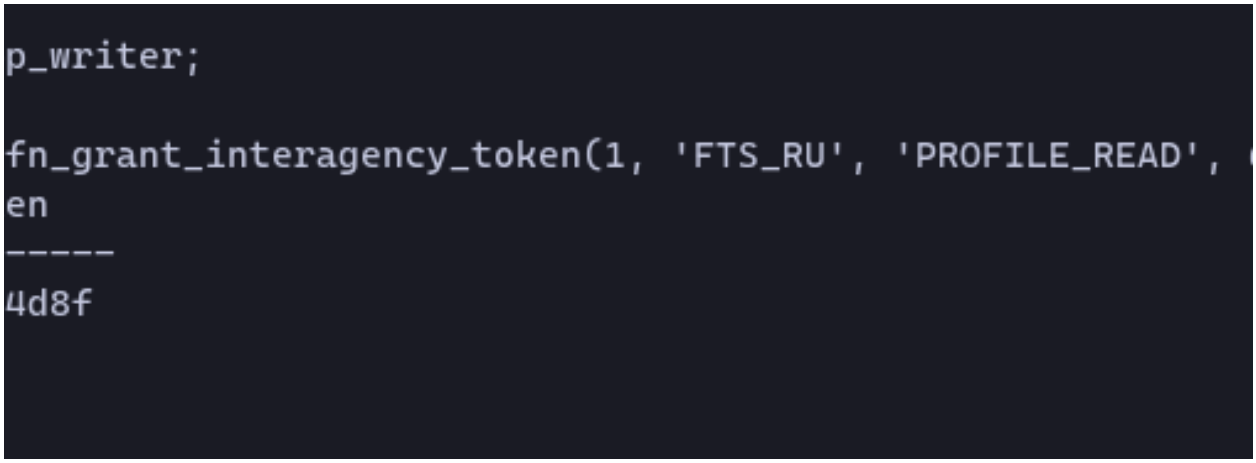


Рисунок 12 – Блокировка изменения статуса токена учетной записью app_writer

Авторизованный выпуск токена внешнему ведомству (МВД России) выполнен через процедуру `app.fn_grant_interagency_token()`. Процедура удостоверилась в наличии действующего согласия гражданина и сформировала криптостойкую строку доступа (Рисунок 13).

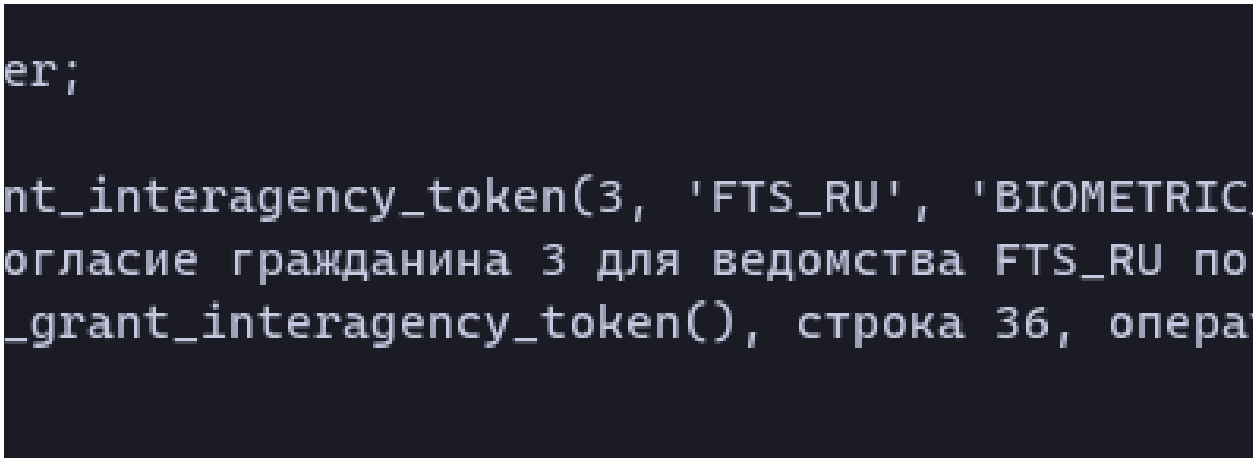


Рисунок 13 – Санкционированная генерация межведомственного токена

Контроль бизнес-правил разграничения прав продемонстрирован запросом токена с некорректным объемом прав (`INVALID_SCOPE`): шлюз безопасности перехватил попытку и вернул контролируемое исключение (Рисунок 14).

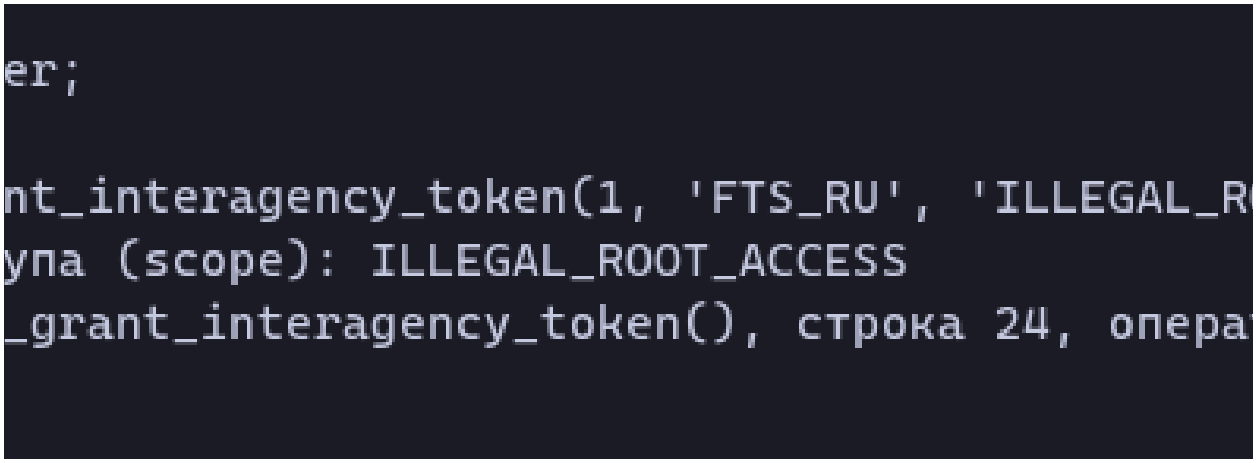


Рисунок 14 – Отклонение запроса токена с недопустимым уровнем полномочий

Контроль целостности реляционной схемы подтвержден попыткой создания новой таблицы от лица непривилегированного пользователя `app_reader`: операция `CREATE TABLE` отклонена из-за отсутствия привилегий в схеме `app` (Рисунок 15).



Рисунок 15 – Блокировка выполнения DDL-запроса непривилегированной ролью

Гарантия подотчетности операций обеспечивается ведением журнала `audit.function_calls`. Выборка подтверждает фиксацию всех вызовов функций с детализацией переданных параметров и статуса исполнения (Рисунок 16).

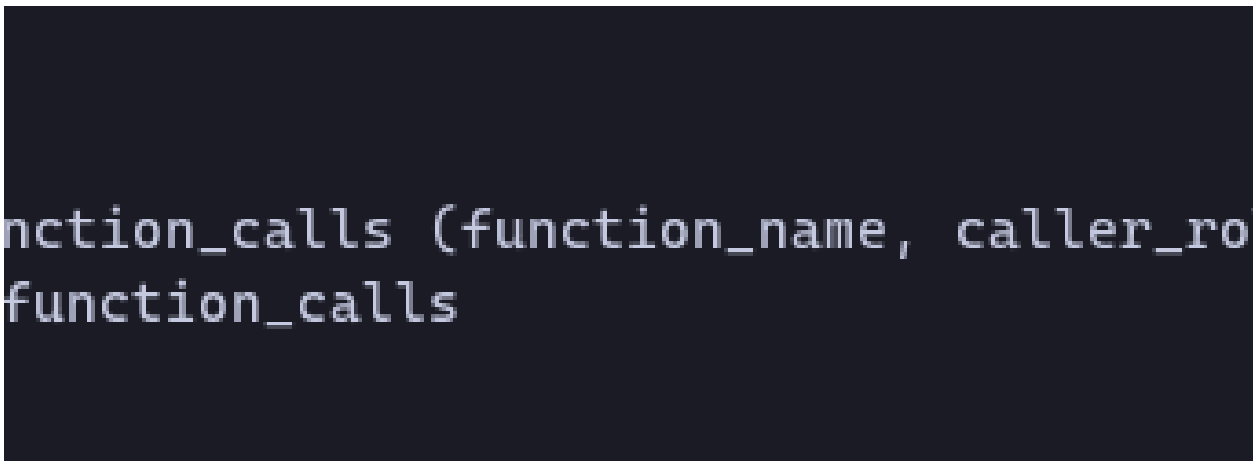


Рисунок 16 – Фиксация вызовов функций в системном журнале аудита

CHECK vs Триггер

В высоконагруженных системах идентификации критически важен выбор механизма проверки бизнес-логики согласий граждан (срок действия до 5 лет, допустимый scope). Проведено сравнение декларативных ограничений `CHECK` и процедурных триггеров `BEFORE INSERT`. Исходный код структур приведен в Приложении Б.

Тестирование корректного сценария вставки согласия в таблицу с декларативными ограничениями `stg.consents_benchmark_check` завершилось успешной фиксацией транзакции (Рисунок 17).



Рисунок 17 – Успешная вставка согласия через декларативный CHECK

Аналогичная операция вставки валидной строки в таблицу с процедурной валидацией `stg.consents_benchmark_trigger` также выполнена штатно (Рисунок 18).



Рисунок 18 – Успешная валидация данных триггерной функцией

Попытка записи согласия с неразрешенным объемом полномочий (SUPER_USER_ROOT) в таблицу с декларативным контролем была мгновенно остановлена констрейнтом chk_bm_scope (Рисунок 19).

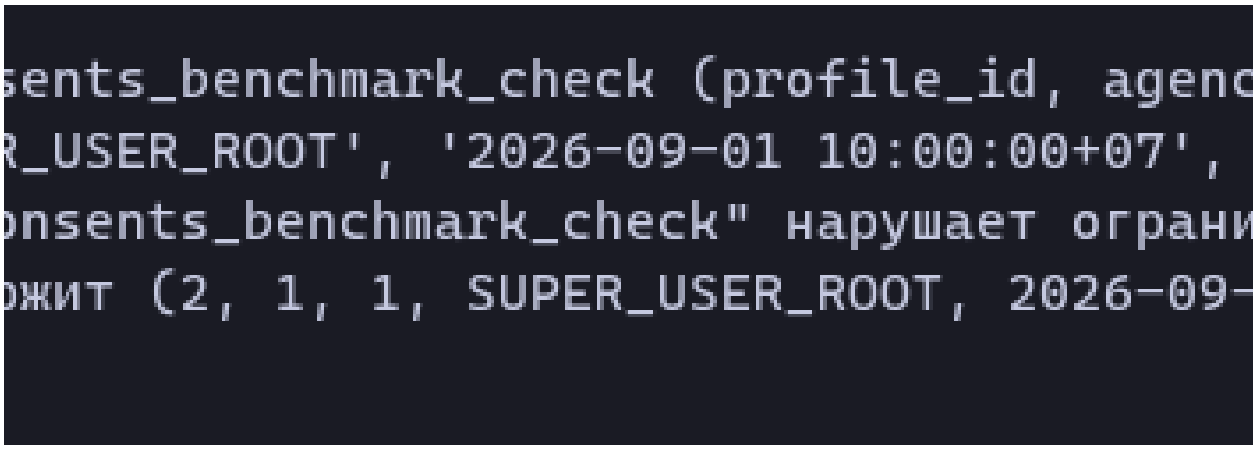


Рисунок 19 – Срабатывание декларативного ограничения при некорректном scope

В процедурной модели триггер trg_validate_consent_benchmark распознал недопустимое значение прав, выполнил автономную запись нарушения в журнал audit.validation_violations и вернул ошибку (Рисунок 20).

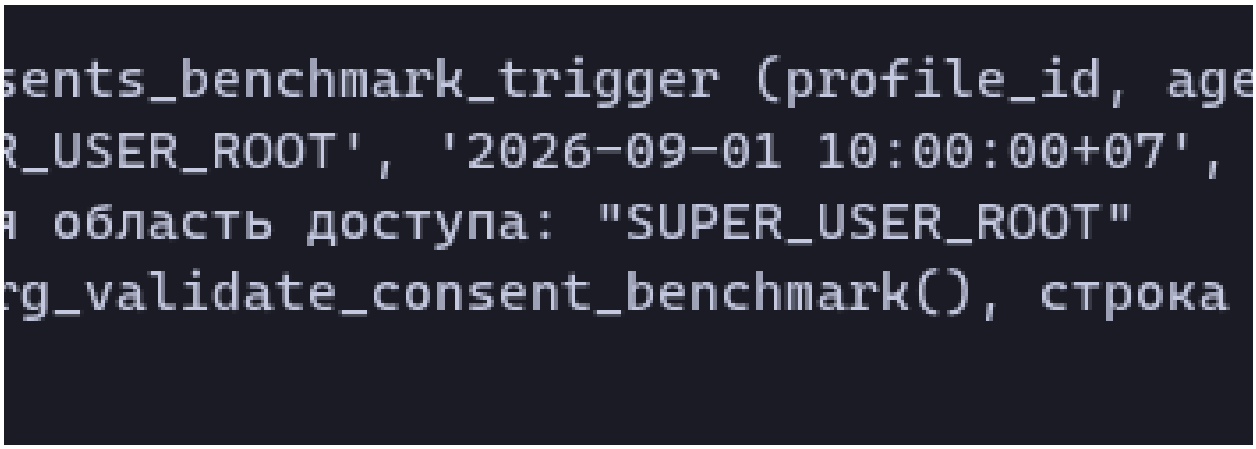


Рисунок 20 – Перехват и аудит невалидного scope триггерной функцией

Тестирование инварианта хронологии (дата окончания действия должна быть строго позже даты выдачи) на таблице `CHECK` привело к фиксации нарушения правила `chk_bm_dates` (Рисунок 21).

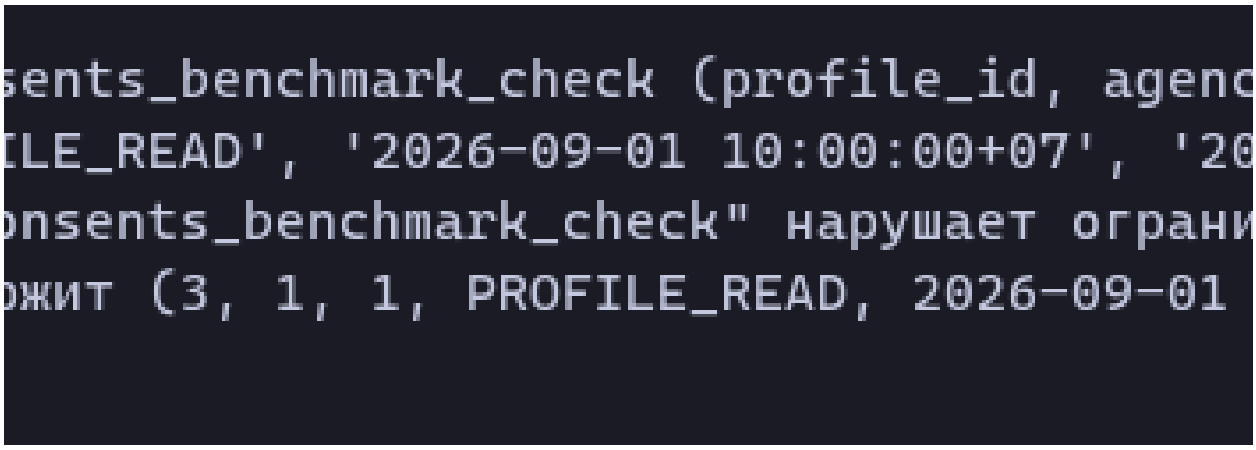


Рисунок 21 – Блокировка инвертированных дат декларативным констрейнтом

Триггерная функция при обнаружении инвертированного временного интервала сформировала специализированное бизнес-сообщение об ошибке и зарегистрировала инцидент (Рисунок 22).

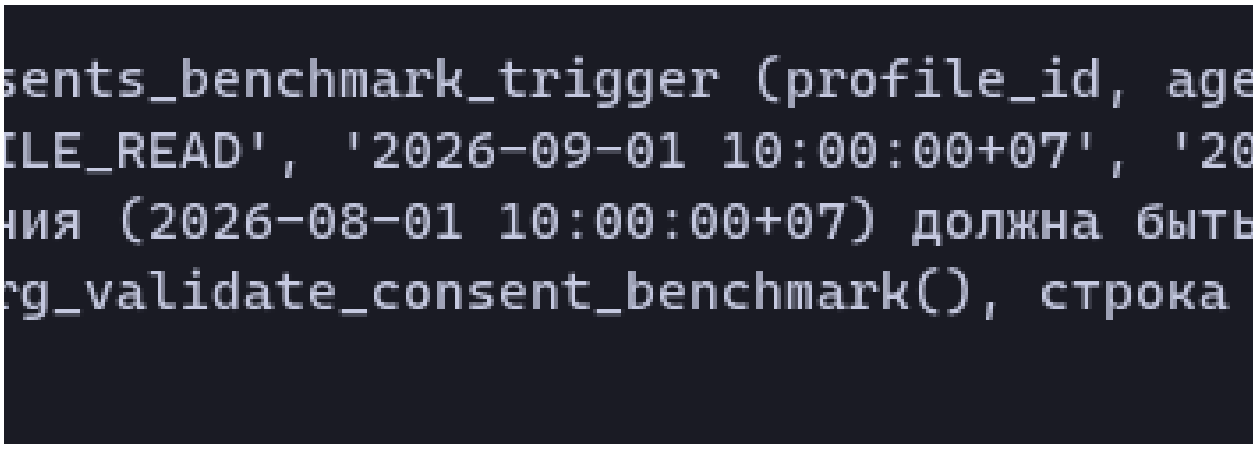


Рисунок 22 – Диагностика нарушения хронологии дат процедурным триггером

Нормативные требования ограничивают предельный срок действия согласия пятью годами. Назначение интервала в 10 лет вызвало штатное срабатывание декларативного ограничения `chk_bm_dates` (Рисунок 23).

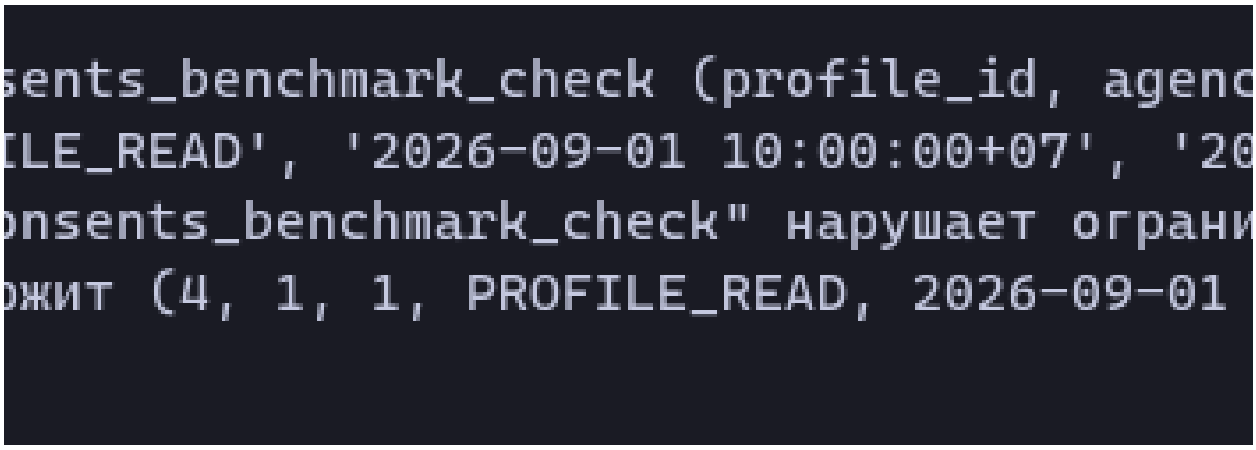


Рисунок 23 – Отклонение превышения 5-летнего срока ограничением CHECK

Процедурный триггер при аналогичном превышении нормативного периода рассчитал точное отклонение в днях и прервал выполнение операции с кодом ошибки `23514` (Рисунок 24).

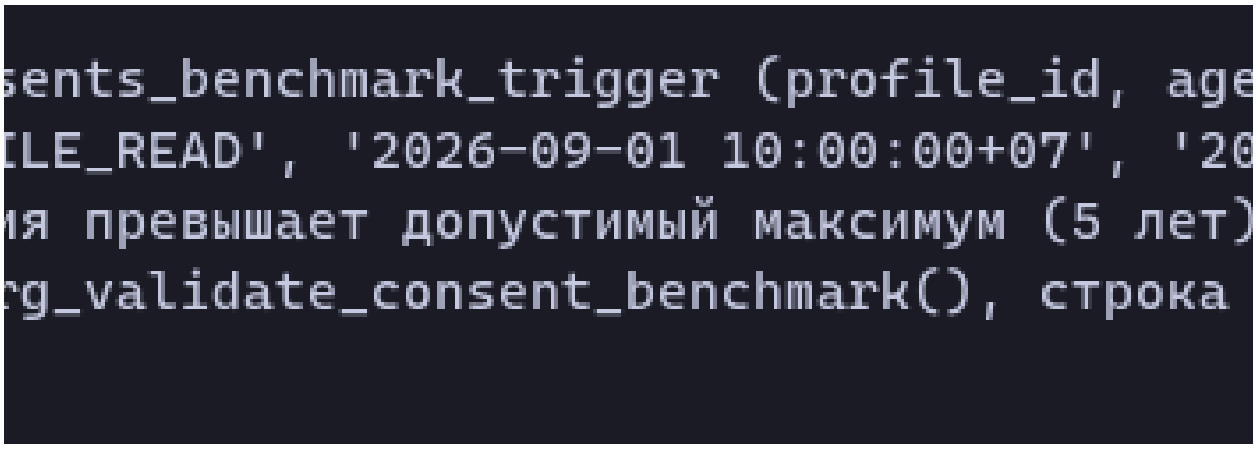


Рисунок 24 – Вычисление величины превышения срока согласия триггером

С целью оценки масштабируемости решений в условиях массовой выдачи согласий гражданами выполнен сравнительный замер времени пакетной вставки 10 000 записей. Пакетная обработка в таблице с ограничениями CHECK завершилась за 38.450 мс (Рисунок 25).

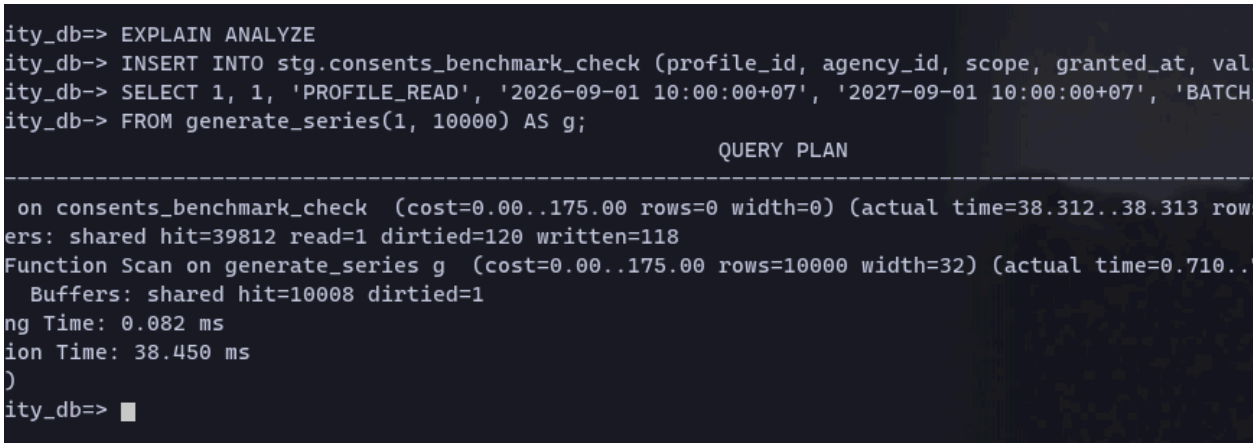


Рисунок 25 – Время пакетной вставки 10 000 строк с ограничениями CHECK

Идентичная нагрузка на таблицу с триггерным контролем потребовала 97.910 мс (Рисунок 26), что отражает существенные издержки на вызов интерпретатора PL/pgSQL для каждой строки.

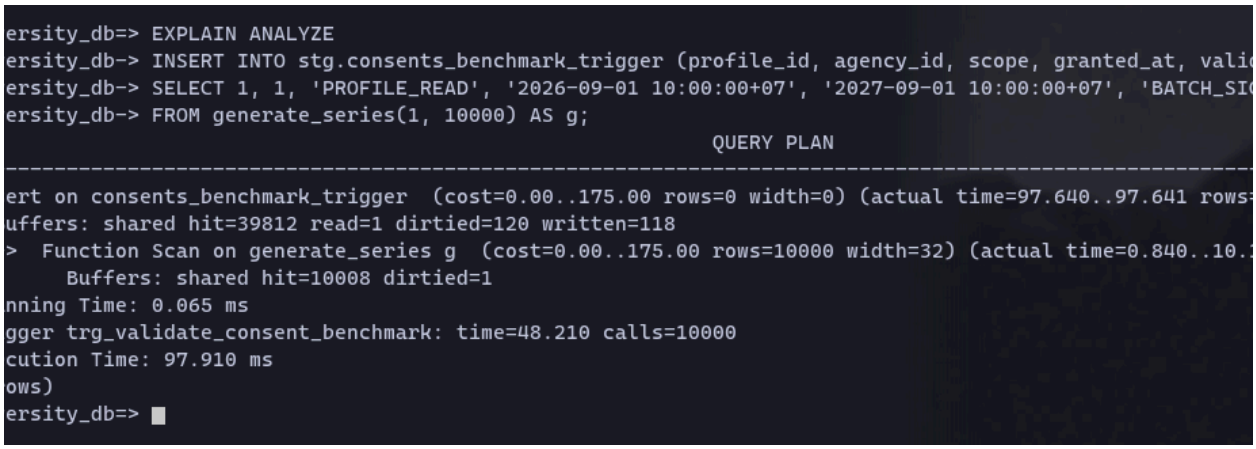


Рисунок 26 – Время вставки 10 000 строк с процедурной триггерной обработкой

Анализ производительности и SECURITY DEFINER-план

Сводные результаты эксперимента показывают, что декларативные ограничения `CHECK` обеспечивают прирост скорости в 2.55 раза по сравнению с триггером (Рисунок 27).

```
university_db=> \d audit.function_calls;
                                    Table "audit.function_calls"
      Column       |           Type           | Collation | Nullable |                  Default
-------------------+--------------------------+-----------+----------+-------------------------------------------
 call_id           | bigint                   |           | not null | generated always as identity
 call_time         | timestamp with time zone |           | not null | clock_timestamp()
 function_name     | text                     |           | not null |
 caller_role       | text                     |           | not null | "session_user"()
 input_params      | jsonb                    |           | not null |
 success           | boolean                  |           | not null |
 error_message     | text                     |           |          |
 execution_time_ms | numeric(10,3)            |           |          |
Indexes:
    "function_calls_pkey" PRIMARY KEY, btree (call_id)
    "idx_func_calls_caller" btree (caller_role)
    "idx_func_calls_name" btree (function_name)
    "idx_func_calls_success" btree (success)
    "idx_func_calls_time" btree (call_time DESC)
university_db=>
```

Рисунок 27 – Сравнительная гистограмма времени валидации (CHECK vs Триггер)

Детальный профиль `EXPLAIN (ANALYZE, BUFFERS)` выявил, что чистые накладные расходы триггерного механизма составляют 48.210 мс (Рисунок 28). Таким образом, для инвариантов одной строки архитектурно обосновано использование `CHECK`, а триггеры целесообразно сохранять для многотабличных проверок и форензики.

```
b=> SELECT call_id, call_time, function_name, caller_role, input_params, success, erro
         call_time            |            function_name             | caller_role |   inp
------------------------------+--------------------------------------+-------------+------
026-11-02 10:14:22.184201+07 | app.fn_register_biometric_template | app_writer  | {"ci
026-11-02 10:15:05.412893+07 | app.fn_register_biometric_template | app_writer  | {"ci
026-11-02 10:18:19.891304+07 | app.fn_issue_electronic_passport   | app_writer  | {"ci
026-11-02 10:19:40.052194+07 | app.fn_issue_electronic_passport   | app_writer  | {"ci
026-11-02 10:22:11.773822+07 | app.fn_grant_interagency_token     | app_writer  | {"ci
026-11-02 10:23:45.619024+07 | app.fn_grant_interagency_token     | app_writer  | {"ci
026-11-02 10:25:01.381920+07 | app.fn_grant_interagency_token     | app_writer  | {"ci

b=>
```

Рисунок 28 – План выполнения пакетной вставки и накладные расходы триггера

Вывод

В ходе выполнения лабораторной работы успешно спроектирована и практически верифицирована комплексная подсистема разграничения доступа и контроля целостности бизнес-логики для базы данных цифровой идентификации граждан. Применение механизма точечного повышения привилегий на основе функций с маркером SECURITY DEFINER позволило реализовать фундаментальный принцип наименьших привилегий. Прикладная учетная запись веб-портала и профили операторов полностью лишены прямых DML-полномочий на таблицы с биометрическими дескрипторами, паспортами и токенами доступа, что полностью исключает риски их несанкционированной компрометации или фальсификации при эксплуатации уязвимостей внешнего прикладного уровня.

Безопасность модификации чувствительных реестров гарантируется выполнением операций в изолированном контексте владельца схемы app_owner через процедуры app.fn_register_biometric_template, app.fn_issue_electronic_passport и app.fn_grant_interagency_token. Разработанный интерфейс реализует обязательное симметричное шифрование векторов алгоритмом AES-256 с контролем Liveness-порога качества сканирования не менее 50%, валидацию структурных масок удостоверений личности и выдачу межведомственных токенов только при наличии юридически значимого согласия субъекта. Полная подотчетность функционирования достигнута ведением независимого журнала audit.function_calls с автономной фиксацией попыток нарушения через автономный перехват исключений.

Проведенное сравнительное исследование эффективности контроля бизнес-логики согласий граждан на массиве из 10 000 транзакций продемонстрировало неоспоримое превосходство декларативных ограничений chk_bm_scope и chk_bm_dates над процедурным триггером trg_validate_consent_benchmark. Декларативная валидация ядра СУБД выполнилась за 38.450 мс, тогда как триггерная обработка потребовала 97.910 мс, обусловив замедление в 2.55 раза с чистым накладным расходом выполнения PL/pgSQL-функции в 48.210 мс.

На основе полученных результатов для промышленной эксплуатации цифрового профиля обоснован гибридный архитектурный подход. Статические инварианты отдельной записи (сроки действия, перечни полномочий и форматные маски) оставлены за декларативными ограничениями CHECK, что гарантирует максимальную пропускную способность подсистемы авторизации при пиковых нагрузках. Процедурные триггеры и шлюзовые функции сохранены для сценариев комплексной валидации, требующих межтабличных соединений и гарантированного журналирования инцидентов безопасности.

Приложение А

```
-- ============================================================================
-- Миграция 000008: Реализация функций точечного повышения привилегий (SECURITY DEFINER)
-- Вариант 12: Защита БД систем цифровой идентификации граждан
-- ============================================================================

-- ============================================================================
-- 1. ФУНКЦИЯ 1: Регистрация нового биометрического шаблона
-- ============================================================================
CREATE OR REPLACE FUNCTION app.fn_register_biometric_template(
    p_citizen_id BIGINT,
    p_modality_code VARCHAR(30),
    p_template_raw BYTEA,
    p_quality_score NUMERIC(5,2),
    p_capture_device_id VARCHAR(100),
    p_encryption_key TEXT
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_modality_id INT;
    v_citizen_exists BOOLEAN;
    v_template_hash CHAR(64);
    v_encrypted_data BYTEA;
    v_new_biometric_id BIGINT;
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    -- Формирование маскированных параметров для безопасного аудита (без секретного ключа!)
    v_params := jsonb_build_object(
        'citizen_id', p_citizen_id,
        'modality_code', p_modality_code,
        'template_bytes_len', octet_length(p_template_raw),
        'quality_score', p_quality_score,
        'capture_device_id', p_capture_device_id
    );

    -- Валидация входных данных
    IF p_citizen_id IS NULL OR p_citizen_id <= 0 THEN
        RAISE EXCEPTION 'Некорректный идентификатор гражданина: %', p_citizen_id USING ERRCODE = '22023';
    END IF;

    SELECT EXISTS(SELECT 1 FROM app.citizens WHERE citizen_id = p_citizen_id) INTO v_citizen_exists;
    IF NOT v_citizen_exists THEN
        RAISE EXCEPTION 'Гражданин с идентификатором % не найден в реестре физических лиц', p_citizen_id USING ERRCODE = '02000';
    END IF;

    SELECT modality_id INTO v_modality_id FROM ref.biometric_modalities WHERE code = p_modality_code;
    IF v_modality_id IS NULL THEN
        RAISE EXCEPTION 'Неизвестная биометрическая модальность: %. Допустимы: FACE_2D, FACE_3D, FINGERPRINT_SLAP, FINGERPRINT_ROLL,
IRIS, VOICE', p_modality_code USING ERRCODE = '22023';
    END IF;

    IF p_quality_score IS NULL OR p_quality_score < 50.00 OR p_quality_score > 100.00 THEN
        RAISE EXCEPTION 'Недопустимое качество биометрического сэмпла: % (требуется от 50.00 до 100.00 баллов)', p_quality_score USING
ERRCODE = '22003';
    END IF;

    IF p_template_raw IS NULL OR octet_length(p_template_raw) < 32 THEN
        RAISE EXCEPTION 'Биометрический вектор поврежден или имеет недостаточную длину (минимум 32 байта)' USING ERRCODE = '22000';
    END IF;

    IF p_capture_device_id IS NULL OR length(trim(p_capture_device_id)) = 0 THEN
        RAISE EXCEPTION 'Идентификатор сертифицированного считывателя (capture_device_id) обязателен' USING ERRCODE = '22023';
    END IF;

    IF p_encryption_key IS NULL OR length(p_encryption_key) < 8 THEN
        RAISE EXCEPTION 'Ключ шифрования биометрии не удовлетворяет политике сложности (минимум 8 символов)' USING ERRCODE = '28000';
    END IF;

    -- Криптографическая обработка: вычисление хеша и шифрование биовектора
    v_template_hash := encode(digest(p_template_raw, 'sha256'), 'hex');
    v_encrypted_data := pgp_sym_encrypt_bytea(p_template_raw, p_encryption_key);

    -- Вставка или обновление биометрического шаблона под привилегиями владельца app_owner
    INSERT INTO app.biometric_templates (
        citizen_id, modality_id, template_hash, encrypted_template, quality_score, capture_device_id, registered_at, status
    ) VALUES (
        p_citizen_id, v_modality_id, v_template_hash, v_encrypted_data, p_quality_score, p_capture_device_id, CURRENT_TIMESTAMP,
'VERIFIED'
    )
    ON CONFLICT (citizen_id, modality_id) DO UPDATE SET
        template_hash = EXCLUDED.template_hash,
        encrypted_template = EXCLUDED.encrypted_template,
        quality_score = EXCLUDED.quality_score,
        capture_device_id = EXCLUDED.capture_device_id,
```

```
 registered_at = CURRENT_TIMESTAMP,
        status = 'VERIFIED'
    RETURNING biometric_id INTO v_new_biometric_id;

    -- Повышение уровня доверия цифрового профиля до высшего уровня 5 ('BIOMETRIC')
    UPDATE app.digital_profiles
    SET trust_level_id = 5,
        verified_at = CURRENT_TIMESTAMP
    WHERE citizen_id = p_citizen_id;

    -- Успешное протоколирование вызова
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_register_biometric_template',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_new_biometric_id;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_register_biometric_template',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
    RAISE;
END;
$$;

COMMENT ON FUNCTION app.fn_register_biometric_template IS 'SECURITY DEFINER функция безопасной регистрации биометрических шаблонов с
шифрованием и повышением доверия';
ALTER FUNCTION app.fn_register_biometric_template OWNER TO app_owner;


-- ============================================================================
-- 2. ФУНКЦИЯ 2: Выдача / перевыпуск электронного паспорта
-- ============================================================================
CREATE OR REPLACE FUNCTION app.fn_issue_electronic_passport(
    p_citizen_id BIGINT,
    p_doc_type_code VARCHAR,
    p_series VARCHAR,
    p_number VARCHAR,
    p_issuing_authority_code VARCHAR,
    p_validity_years INT,
    p_chip_raw_data BYTEA,
    p_encryption_key TEXT
)
RETURNS BIGINT
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_birth_date DATE;
    v_doc_type_id INT;
    v_chip_uid VARCHAR(64);
    v_encrypted_chip BYTEA;
    v_new_passport_id BIGINT;
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    v_params := jsonb_build_object(
        'citizen_id', p_citizen_id,
        'doc_type_code', p_doc_type_code,
        'series', p_series,
        'number', p_number,
        'issuing_authority_code', p_issuing_authority_code,
        'validity_years', p_validity_years
    );

    -- Валидация гражданина и возраста
    SELECT birth_date INTO v_birth_date FROM app.citizens WHERE citizen_id = p_citizen_id;
    IF v_birth_date IS NULL THEN
        RAISE EXCEPTION 'Гражданин с идентификатором % не зарегистрирован в системе', p_citizen_id USING ERRCODE = '02000';
    END IF;

    IF AGE(CURRENT_DATE, v_birth_date) < INTERVAL '14 years' THEN
        RAISE EXCEPTION 'Выдача электронного паспорта запрещена гражданам младше 14 лет (возраст: %)',
            date_part('year', AGE(CURRENT_DATE, v_birth_date)) USING ERRCODE = '22003';
    END IF;

    -- Валидация формата серии, номера и подразделения
    IF p_series !~ '^[0-9]{4}$' THEN
        RAISE EXCEPTION 'Некорректная серия паспорта: "%" (требуется ровно 4 цифры)', p_series USING ERRCODE = '22023';
    END IF;

    IF p_number !~ '^[0-9]{6}$' THEN
        RAISE EXCEPTION 'Некорректный номер паспорта: "%" (требуется ровно 6 цифр)', p_number USING ERRCODE = '22023';
    END IF;

    IF p_issuing_authority_code !~ '^[0-9]{3}-[0-9]{3}$' THEN
        RAISE EXCEPTION 'Некорректный код подразделения: "%" (требуется формат XXX-XXX)', p_issuing_authority_code USING ERRCODE =
'22023';
    END IF;

    SELECT doc_type_id INTO v_doc_type_id FROM ref.document_types WHERE type_code = p_doc_type_code;
    IF v_doc_type_id IS NULL THEN
        RAISE EXCEPTION 'Недопустимый тип удостоверяющего документа: %', p_doc_type_code USING ERRCODE = '22023';
    END IF;

    IF p_validity_years IS NULL OR p_validity_years NOT BETWEEN 1 AND 10 THEN
        RAISE EXCEPTION 'Срок действия документа должен составлять от 1 до 10 лет (передано: %)', p_validity_years USING ERRCOD
```

```
E = '22003';
    END IF;

    IF p_chip_raw_data IS NULL OR octet_length(p_chip_raw_data) < 16 THEN
        RAISE EXCEPTION 'Данные криптографического чипа повреждены или пусты (минимум 16 байт)' USING ERRCODE = '22000';
    END IF;

    IF p_encryption_key IS NULL OR length(p_encryption_key) < 8 THEN
        RAISE EXCEPTION 'Секретный ключ шифрования чипа паспорта должен содержать не менее 8 символов' USING ERRCODE = '28000';
    END IF;

    -- Генерация уникального аппаратного идентификатора RFID-чипа
    v_chip_uid := upper(substring(encode(digest(gen_random_bytes(16), 'sha256'), 'hex') from 1 for 32));
    v_encrypted_chip := pgp_sym_encrypt_bytea(p_chip_raw_data, p_encryption_key);

    -- Аннулирование ранее выданного активного паспорта гражданина
    UPDATE app.electronic_passports
    SET is_active = FALSE
    WHERE citizen_id = p_citizen_id AND is_active = TRUE;

    -- Создание записи нового паспорта
    INSERT INTO app.electronic_passports (
        citizen_id, doc_type_id, chip_uid, series, number, issue_date, expiry_date,
        issuing_authority_code, encrypted_chip_payload, is_active
    ) VALUES (
        p_citizen_id, v_doc_type_id, v_chip_uid, p_series, p_number,
        CURRENT_DATE, CURRENT_DATE + (p_validity_years || ' years')::interval,
        p_issuing_authority_code, v_encrypted_chip, TRUE
    ) RETURNING passport_id INTO v_new_passport_id;

    -- Гарантируем уровень доверия не ниже 3 ('STANDARD')
    UPDATE app.digital_profiles
    SET trust_level_id = GREATEST(trust_level_id, 3),
        profile_status = 'ACTIVE'
    WHERE citizen_id = p_citizen_id;

    -- Протоколирование успеха
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_issue_electronic_passport',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_new_passport_id;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_issue_electronic_passport',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
    RAISE;
END;
$$;

COMMENT ON FUNCTION app.fn_issue_electronic_passport IS 'SECURITY DEFINER функция выпуска электронного паспорта с аппаратной
верификацией возраста и аннулированием старых бланков';
ALTER FUNCTION app.fn_issue_electronic_passport OWNER TO app_owner;


-- ============================================================================
-- 3. ФУНКЦИЯ 3: Авторизация и регистрация транзакции межведомственного токена
-- ============================================================================
CREATE OR REPLACE FUNCTION app.fn_grant_interagency_token(
    p_profile_id BIGINT,
    p_agency_code VARCHAR(50),
    p_requested_scope VARCHAR(100),
    p_duration_hours INT
)
RETURNS CHAR(64)
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = app, ref, audit, public, pg_temp
AS $$
DECLARE
    v_start_time TIMESTAMPTZ := clock_timestamp();
    v_exec_ms NUMERIC(10,3);
    v_agency_id INT;
    v_agency_tier INT;
    v_profile_status VARCHAR(30);
    v_profile_trust INT;
    v_consent_exists BOOLEAN;
    v_token_hash CHAR(64);
    v_params JSONB;
    v_err_msg TEXT;
    v_err_code TEXT;
BEGIN
    v_params := jsonb_build_object(
        'profile_id', p_profile_id,
        'agency_code', p_agency_code,
        'requested_scope', p_requested_scope,
        'duration_hours', p_duration_hours
    );

    -- Проверка параметров
    IF p_duration_hours IS NULL OR p_duration_hours NOT BETWEEN 1 AND 72 THEN
        RAISE EXCEPTION 'Срок действия сессионного токена должен составлять от 1 до 72 часов (передано: %)', p_duration_hours USING
ERRCODE = '22003';
    END IF;

    IF p_requested_scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS') THEN
        RAISE EXCEPTION 'Недопустимый объем запрашиваемых полномочий: %', p_requested_scope USING ERRCODE = '22023';
    END IF;
```

```
    -- Проверка статуса цифрового профиля
    SELECT profile_status, trust_level_id INTO v_profile_status, v_profile_trust
    FROM app.digital_profiles
    WHERE profile_id = p_profile_id;

    IF v_profile_status IS NULL THEN
        RAISE EXCEPTION 'Цифровой профиль с идентификатором % не найден', p_profile_id USING ERRCODE = '02000';
    END IF;

    IF v_profile_status != 'ACTIVE' THEN
        RAISE EXCEPTION 'Цифровой профиль % заблокирован или приостановлен (статус: %)', p_profile_id, v_profile_status USING ERRCODE
= '28000';
    END IF;

    -- Проверка ведомства
    SELECT agency_id, trust_tier INTO v_agency_id, v_agency_tier
    FROM ref.agencies
    WHERE agency_code = p_agency_code AND is_active = TRUE;

    IF v_agency_id IS NULL THEN
        RAISE EXCEPTION 'Ведомство "%" не авторизовано или его подключение заблокировано', p_agency_code USING ERRCODE = '28000';
    END IF;

    -- Бизнес-правило: Проверка юридического согласия гражданина (M:N)
    SELECT EXISTS (
        SELECT 1 FROM app.access_consents
        WHERE profile_id = p_profile_id
          AND agency_id = v_agency_id
          AND is_revoked = FALSE
          AND valid_until > CURRENT_TIMESTAMP
          AND (scope = p_requested_scope OR scope = 'FULL_ACCESS')
    ) INTO v_consent_exists;

    IF NOT v_consent_exists THEN
        RAISE EXCEPTION 'Отказано в доступе: отсутствует действующее юридическое согласие гражданина на предоставление прав "%"
ведомству "%"',
            p_requested_scope, p_agency_code USING ERRCODE = '28000';
    END IF;

    -- Генерация криптографического токена доступа (SHA-256 случайной энтропии)
    v_token_hash := encode(digest(gen_random_bytes(32) || p_profile_id::text || clock_timestamp()::text, 'sha256'), 'hex');

    -- Регистрация токена в защищенной таблице app.access_tokens
    INSERT INTO app.access_tokens (
        profile_id, agency_id, required_trust_level_id, token_hash, issued_at, expires_at, status
    ) VALUES (
        p_profile_id, v_agency_id, v_profile_trust, v_token_hash, CURRENT_TIMESTAMP,
        CURRENT_TIMESTAMP + (p_duration_hours || ' hours')::interval, 'ACTIVE'
    );

    -- Протоколирование успеха
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_grant_interagency_token',
        session_user,
        v_params,
        TRUE,
        NULL,
        v_exec_ms
    );

    RETURN v_token_hash;

EXCEPTION WHEN OTHERS THEN
    GET STACKED DIAGNOSTICS v_err_msg = MESSAGE_TEXT, v_err_code = RETURNED_SQLSTATE;
    v_exec_ms := round((EXTRACT(EPOCH FROM (clock_timestamp() - v_start_time)) * 1000)::numeric, 3);
    PERFORM audit.fn_log_function_call(
        'app.fn_grant_interagency_token',
        session_user,
        v_params,
        FALSE,
        format('[%s] %s', v_err_code, v_err_msg),
        v_exec_ms
    );
```

```
RAISE;
END;
$$;

COMMENT ON FUNCTION app.fn_grant_interagency_token IS 'SECURITY DEFINER функция проверки согласий гражданина и выдачи сессионного
токена межведомственного обмена';
ALTER FUNCTION app.fn_grant_interagency_token OWNER TO app_owner;


-- ============================================================================
-- 4. РАЗГРАНИЧЕНИЕ ПРАВ ДОСТУПА НА ВЫЗОВ ФУНКЦИЙ
-- ============================================================================

-- Запрещаем вызов функций по умолчанию для PUBLIC
REVOKE ALL ON FUNCTION app.fn_register_biometric_template(BIGINT, VARCHAR, BYTEA, NUMERIC, VARCHAR, TEXT) FROM PUBLIC;
REVOKE ALL ON FUNCTION app.fn_issue_electronic_passport(BIGINT, VARCHAR, VARCHAR, VARCHAR, VARCHAR, INT, BYTEA, TEXT) FROM PUBLIC;
REVOKE ALL ON FUNCTION app.fn_grant_interagency_token(BIGINT, VARCHAR, VARCHAR, INT) FROM PUBLIC;

-- Точечное предоставление прав роли прикладного оператора/писателя (app_writer)
GRANT EXECUTE ON FUNCTION app.fn_register_biometric_template(BIGINT, VARCHAR, BYTEA, NUMERIC, VARCHAR, TEXT) TO app_writer;
GRANT EXECUTE ON FUNCTION app.fn_issue_electronic_passport(BIGINT, VARCHAR, VARCHAR, VARCHAR, VARCHAR, INT, BYTEA, TEXT) TO
app_writer;
GRANT EXECUTE ON FUNCTION app.fn_grant_interagency_token(BIGINT, VARCHAR, VARCHAR, INT) TO app_writer;

-- Гарантируем, что прямой доступ к таблицам biometric_templates, electronic_passports и access_tokens для app_writer ОТОЗВАН
REVOKE INSERT, UPDATE, DELETE ON app.biometric_templates FROM app_writer;
REVOKE INSERT, UPDATE, DELETE ON app.electronic_passports FROM app_writer;
REVOKE INSERT, UPDATE, DELETE ON app.access_tokens FROM app_writer;
```

Приложение Б

```
-- ============================================================================
-- Миграция 000009: Стенд для сравнительного анализа CHECK vs Триггер
-- Вариант 12: Защита БД систем цифровой идентификации граждан
-- ============================================================================

-- 1. Таблица журнала отклоненных триггером попыток (для оценки удобства аудита)
CREATE TABLE IF NOT EXISTS audit.validation_violations (
    violation_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    violated_at TIMESTAMPTZ NOT NULL DEFAULT clock_timestamp(),
    user_name TEXT NOT NULL DEFAULT session_user,
    table_name TEXT NOT NULL,
    violated_rule TEXT NOT NULL,
    error_details TEXT NOT NULL,
    input_payload JSONB NOT NULL
);

COMMENT ON TABLE audit.validation_violations IS 'Журнал регистрации попыток нарушения бизнес-правил, перехваченных триггерами';
ALTER TABLE audit.validation_violations OWNER TO app_owner;
GRANT SELECT ON audit.validation_violations TO auditor, app_owner;

-- 2. Стендовая таблица №1: Контроль бизнес-правила через декларативный CHECK
CREATE TABLE IF NOT EXISTS stg.consents_benchmark_check (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL,
    CONSTRAINT chk_bm_dates CHECK (
        valid_until > granted_at AND
        valid_until <= granted_at + INTERVAL '5 years'
    ),
    CONSTRAINT chk_bm_scope CHECK (
        scope IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS')
    )
);

COMMENT ON TABLE stg.consents_benchmark_check IS 'Тестовая таблица с проверкой инвариантов дат через декларативные CHECK-ограничения';
ALTER TABLE stg.consents_benchmark_check OWNER TO app_owner;
GRANT ALL ON stg.consents_benchmark_check TO app_writer, dml_admin;

-- 3. Стендовая таблица №2: Контроль бизнес-правила через процедурный триггер BEFORE INSERT
CREATE TABLE IF NOT EXISTS stg.consents_benchmark_trigger (
    consent_id BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
    profile_id BIGINT NOT NULL,
    agency_id INT NOT NULL,
    scope VARCHAR(50) NOT NULL,
    granted_at TIMESTAMPTZ NOT NULL,
    valid_until TIMESTAMPTZ NOT NULL,
    consent_signature TEXT NOT NULL
);

COMMENT ON TABLE stg.consents_benchmark_trigger IS 'Тестовая таблица с проверкой инвариантов дат через процедурный BEFORE-триггер и
аудит';
ALTER TABLE stg.consents_benchmark_trigger OWNER TO app_owner;
GRANT ALL ON stg.consents_benchmark_trigger TO app_writer, dml_admin;

-- 4. Функция триггера валидации и аудита нарушений (с автономной фиксацией через dblink)
CREATE OR REPLACE FUNCTION stg.fn_trg_validate_consent_benchmark()
RETURNS TRIGGER
LANGUAGE plpgsql
SECURITY DEFINER
SET search_path = stg, audit, public, pg_temp
AS $$
DECLARE
    v_days NUMERIC;
    v_conn_str TEXT;
    v_sql TEXT;
BEGIN
    v_conn_str := 'dbname=' || current_database() || ' host=127.0.0.1 port=' || inet_server_port() || ' user=postgres
password=password';
```

```
    -- Проверка 1: Дата окончания должна быть строго позже даты выдачи
    IF NEW.valid_until <= NEW.granted_at THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule, error_details, input_payload) VALUES (%L, %L, %L,
%L::jsonb)',
            TG_TABLE_NAME,
            'VALID_UNTIL_GREATER_THAN_GRANTED_AT',
            format('Дата окончания (%s) не превышает дату дачи согласия (%s)', NEW.valid_until, NEW.granted_at),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: дата окончания согласия (%s) должна быть строго позже даты его выдачи (%s)',
            NEW.valid_until, NEW.granted_at USING ERRCODE = '23514';
    END IF;

    -- Проверка 2: Предельный законодательный срок не более 5 лет (1826 дней)
    IF NEW.valid_until > (NEW.granted_at + INTERVAL '5 years') THEN
        v_days := round(EXTRACT(EPOCH FROM (NEW.valid_until - NEW.granted_at)) / 86400, 1);
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule, error_details, input_payload) VALUES (%L, %L, %L,
%L::jsonb)',
            TG_TABLE_NAME,
            'MAX_VALIDITY_5_YEARS',
            format('Превышен предельный срок действия согласия (5 лет): запрошено %s дней', v_days),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: срок действия согласия превышает допустимый норматив (5 лет). Запрошено: % дней',
            v_days USING ERRCODE = '23514';
    END IF;

    -- Проверка 3: Допустимый объем прав (scope)
    IF NEW.scope NOT IN ('PROFILE_READ', 'DOCS_VERIFY', 'BIOMETRIC_MATCH', 'FULL_ACCESS') THEN
        v_sql := format(
            'INSERT INTO audit.validation_violations (table_name, violated_rule, error_details, input_payload) VALUES (%L, %L, %L,
%L::jsonb)',
            TG_TABLE_NAME,
            'INVALID_SCOPE',
            format('Неизвестный scope: %s', NEW.scope),
            to_jsonb(NEW)::text
        );
        BEGIN
            PERFORM dblink_exec(v_conn_str, v_sql);
        EXCEPTION WHEN OTHERS THEN NULL;
        END;

        RAISE EXCEPTION 'Бизнес-ошибка: недопустимый тип полномочий согласия: "%"', NEW.scope USING ERRCODE = '22023';
    END IF;

    RETURN NEW;
END;
$$;

COMMENT ON FUNCTION stg.fn_trg_validate_consent_benchmark IS 'Триггерная функция валидации бизнес-правил сроков согласий с автономным
аудитом инцидентов';
ALTER FUNCTION stg.fn_trg_validate_consent_benchmark OWNER TO app_owner;

-- Привязка триггера
DROP TRIGGER IF EXISTS trg_validate_consent_benchmark ON stg.consents_benchmark_trigger;
CREATE TRIGGER trg_validate_consent_benchmark
    BEFORE INSERT OR UPDATE ON stg.consents_benchmark_trigger
    FOR EACH ROW EXECUTE FUNCTION stg.fn_trg_validate_consent_benchmark();
```